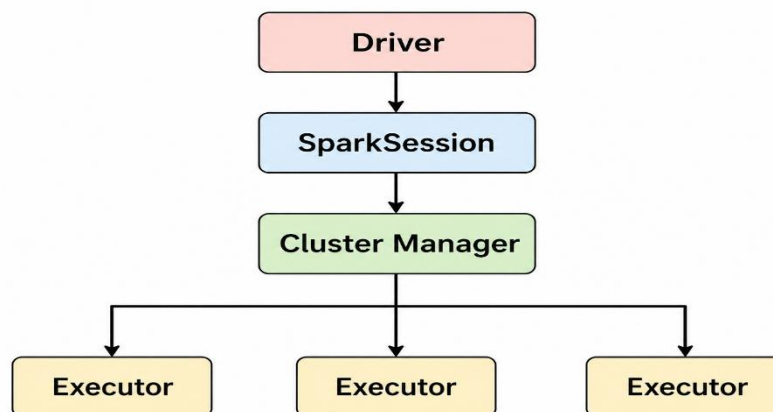


WEEK 6 ASSIGNMENT

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Ans:

Component	Role
Driver	The Driver is the main program of a Spark application. It creates the SparkSession, converts the user code into tasks, schedules those tasks, and collects the results from the executors.
Cluster Manager	The Cluster Manager is responsible for allocating resources (CPU and memory) to the Spark application and managing the available worker nodes. Examples include Standalone, YARN, and Kubernetes.
Executor	Executors are worker processes that run the tasks assigned by the Driver. They process the data, perform computations, store intermediate results in memory or disk, and send the results back to the Driver.



Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Ans:

Spark uses **Lazy Evaluation**, which means it **does not execute transformations immediately**. Instead, it records all transformations (such as `filter()`, `select()`, and `withColumn()`) in a **Directed Acyclic Graph (DAG)**. The transformations are executed only when an **action** (such as `show()`, `count()`, or `write()`) is called.

Before execution, Spark analyzes the DAG and optimizes the execution plan by combining operations, removing unnecessary computations, and reducing data movement. This results in faster execution and better resource utilization, especially for large datasets.

Example:

```
age_df = df.filter(col("age") > 30)
```

```
age_df.show()
```

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username	status	raw_timestamp
1014	14-01-2025	West	Furniture	165	Delhi	31	Basic	165		S101user1014@gmail.com	user1014	Completed	14-01-2025 10:13
1015	15-01-2025	West	Clothing	170	Delhi	32	Premium	170		S101user1015@gmail.com	user1015	Completed	15-01-2025 10:14
1016	16-01-2025	West	Grocery	175	Delhi	33	Basic	175		S101user1016@gmail.com	user1016	Completed	16-01-2025 10:15
1017	17-01-2025	West	Electronics	180	Delhi	34	Premium	180		S101user1017@gmail.com	user1017	Completed	17-01-2025 10:16
1018	18-01-2025	West	Furniture	185	Delhi	35	Basic	NULL		S101user1018@gmail.com	user1018	Completed	18-01-2025 10:17
1032	04-01-2025	West	Grocery	255	Delhi	31	Basic	255		S101user1032@gmail.com	user1032	Completed	04-01-2025 10:31
1033	05-01-2025	West	Electronics	260	Delhi	32	Premium	260		S101user1033@gmail.com	user1033	Completed	05-01-2025 10:32
1034	06-01-2025	West	Furniture	265	Delhi	33	Basic	265		S101user1034@gmail.com	user1034	Completed	06-01-2025 10:33
1035	07-01-2025	West	Clothing	270	Delhi	34	Premium	NULL		S101user1035@gmail.com	user1035	Completed	07-01-2025 10:34
1036	08-01-2025	West	Grocery	275	Delhi	35	Basic	275		S101user1036@gmail.com	user1036	Completed	08-01-2025 10:35
1050	22-01-2025	West	Furniture	345	Delhi	31	Basic	345		S101user1050@gmail.com	user1050	Completed	22-01-2025 10:49
1051	23-01-2025	West	Clothing	350	Delhi	32	Premium	350		S101user1051@gmail.com	user1051	Completed	23-01-2025 10:50
1052	24-01-2025	West	Grocery	355	Delhi	33	Basic	NULL		S101user1052@gmail.com	user1052	Completed	24-01-2025 10:51
1053	25-01-2025	West	Electronics	360	Delhi	34	Premium	360		S101user1053@gmail.com	user1053	Completed	25-01-2025 10:52
1054	26-01-2025	West	Furniture	365	Delhi	35	Basic	365		S101user1054@gmail.com	user1054	Completed	26-01-2025 10:53
1068	12-01-2025	West	Grocery	435	Delhi	31	Basic	435		S101user1068@gmail.com	user1068	Completed	12-01-2025 10:07
1069	13-01-2025	West	Electronics	440	Delhi	32	Premium	NULL		S101user1069@gmail.com	user1069	Completed	13-01-2025 10:08
1070	14-01-2025	West	Furniture	445	Delhi	33	Basic	445		S101user1070@gmail.com	NULL	Completed	14-01-2025 10:09
1071	15-01-2025	West	Clothing	450	Delhi	34	Premium	450		S101user1071@gmail.com	user1071	Completed	15-01-2025 10:10
1072	16-01-2025	West	Grocery	455	Delhi	35	Basic	455		S101user1072@gmail.com	user1072	Completed	16-01-2025 10:11

only showing top 20 rows

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Ans:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
```

```
    .appName("ReadCSV") \
```

```
    .getOrCreate()
```

```
df = spark.read.csv(
```

```
    "data/source.csv",
```

```
    header=True,
```

```
    inferSchema=True
```

```
)
```

```
df.show()
```

Explanation

- `spark.read.csv()` → Reads the CSV file.

- **"data/source.csv"** → Path of the CSV file.
- **header=True** → Uses the first row as column names.
- **inferSchema=True** → Automatically detects the data types of the columns.
- **df.show()** → Displays the first 20 rows of the DataFrame.

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans:

Feature	CSV	Parquet
Storage Format	Row-based (stores complete rows together)	Columnar (stores values of the same column together)
File Size	Larger	Smaller (compressed)
Read Performance	Slower	Faster
Schema	Not stored	Stored with the file
Predicate Pushdown	Not supported	Supported
Best Use	Data exchange and simple storage	Big data analytics and Spark processing

Why does it matter for performance?

- **CSV** stores data row by row, so Spark must read the entire row even if only one column is needed. This increases disk I/O and processing time.
- **Parquet** stores data column by column, allowing Spark to read only the required columns. It also supports compression and Predicate Pushdown, reducing the amount of data loaded into memory and improving query performance.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Ans:

Code

```
from pyspark.sql.functions import col
result_df = df.filter(col("category") == "Electronics") \
    .select("product_id", "price")
result_df.show()
```

```
+-----+-----+
|user_id|price|
+-----+-----+
|    1001| NULL|
|    1005|  120|
|    1009|  140|
|    1013|  160|
|    1017|  180|
|    1021|  200|
|    1025|  220|
|    1029|  240|
|    1033|  260|
|    1037|  280|
|    1041|  300|
|    1045|  320|
|    1049|  340|
|    1053|  360|
|    1057|  380|
|    1061|  400|
|    1065|  420|
|    1069| NULL|
|    1073|  460|
|    1077|  480|
+-----+-----+
only showing top 20 rows
```

Explanation

- `filter(col("category") == "Electronics")` → Filters rows where the category is **Electronics**.
- `select("product_id", "price")` → Selects only the `product_id` and `price` columns.
- `show()` → Displays the filtered result.

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

Ans:

```
from pyspark.sql.functions import col
```

```
# Rename the column
```

```
df = df.withColumnRenamed(
```

```
    "sale_amount",
```

```
    "total_sale"
```

```
)
```

```
# Cast Age from String/Integer to Double
```

```
df = df.withColumn(
```

```
    "age",
```

```
    col("age").cast("double")
```

```
)
```

```
df.show()
```

Output:

user_id	transaction_date	region	product_category	total_sale	city	age	subscription	price	store_id	email	username	status	raw_timestamp	final_price
1001	01-01-2025	West	Electronics	100	Delhi	18.0	Premium	NULL	S101	NULL	NULL	NULL	01-01-2025 10:00	NULL
1002	02-01-2025	West	Furniture	105	Delhi	19.0	Basic	105	S101	user1002@gmail.com	user1002	Completed	02-01-2025 10:01	115.50000000000001
1003	03-01-2025	West	Clothing	110	Delhi	20.0	Premium	110	S101	user1003@gmail.com	user1003	Completed	03-01-2025 10:02	121.00000000000001
1004	04-01-2025	West	Grocery	115	Delhi	21.0	Basic	115	S101	user1004@gmail.com	user1004	Completed	04-01-2025 10:03	126.50000000000001
1005	05-01-2025	West	Electronics	120	Delhi	22.0	Premium	120	S101	user1005@gmail.com	user1005	Completed	05-01-2025 10:04	132.0
1006	06-01-2025	West	Furniture	125	Delhi	23.0	Basic	125	S101	user1006@gmail.com	user1006	Completed	06-01-2025 10:05	137.5
1007	07-01-2025	West	Clothing	130	Delhi	24.0	Premium	130	S101	user1007@gmail.com	user1007	Completed	07-01-2025 10:06	143.0
1008	08-01-2025	West	Grocery	135	Delhi	25.0	Basic	135	S101	user1008@gmail.com	user1008	Completed	08-01-2025 10:07	148.5
1009	09-01-2025	West	Electronics	140	Delhi	26.0	Premium	140	S101	user1009@gmail.com	user1009	Completed	09-01-2025 10:08	154.0
1010	10-01-2025	West	Furniture	145	Delhi	27.0	Basic	145	S101	user1010@gmail.com	user1010	Completed	10-01-2025 10:09	159.5
1011	11-01-2025	West	Clothing	150	Delhi	28.0	Premium	150	S101	user1011@gmail.com	user1011	Completed	11-01-2025 10:10	165.0
1012	12-01-2025	West	Grocery	155	Delhi	29.0	Basic	155	S101	user1012@gmail.com	user1012	Completed	12-01-2025 10:11	170.5
1013	13-01-2025	West	Electronics	160	Delhi	30.0	Premium	160	S101	user1013@gmail.com	user1013	Completed	13-01-2025 10:12	176.0
1014	14-01-2025	West	Furniture	165	Delhi	31.0	Basic	165	S101	user1014@gmail.com	user1014	Completed	14-01-2025 10:13	181.50000000000003
1015	15-01-2025	West	Clothing	170	Delhi	32.0	Premium	170	S101	user1015@gmail.com	user1015	Completed	15-01-2025 10:14	187.00000000000003
1016	16-01-2025	West	Grocery	175	Delhi	33.0	Basic	175	S101	user1016@gmail.com	user1016	Completed	16-01-2025 10:15	192.50000000000003
1017	17-01-2025	West	Electronics	180	Delhi	34.0	Premium	180	S101	user1017@gmail.com	user1017	Completed	17-01-2025 10:16	198.00000000000003
1018	18-01-2025	West	Furniture	185	Delhi	35.0	Basic	NULL	S101	user1018@gmail.com	user1018	Completed	18-01-2025 10:17	NULL
1019	19-01-2025	West	Clothing	190	Delhi	18.0	Premium	190	S101	user1019@gmail.com	user1019	Completed	19-01-2025 10:18	209.00000000000003
1020	20-01-2025	West	Grocery	195	Delhi	19.0	Basic	195	S101	NULL	user1020	Completed	20-01-2025 10:19	214.50000000000003

only showing top 20 rows

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Ans:

Spark uses a Lineage Graph (DAG – Directed Acyclic Graph) to provide fault tolerance by tracking all transformations applied to the original data.

Instead of storing intermediate results, Spark remembers the sequence of transformations (like filter, map, select, etc.) used to create a dataset.

If a worker node (executor) fails, Spark does not restart the whole application. Instead, it:

- Uses the Lineage Graph to identify the lost partitions
- Recomputes only the missing data using the original source data
- Re-applies the required transformations step-by-step

This process is called recomputation using lineage, and it ensures:

- Fault tolerance
- Efficient recovery
- No unnecessary recomputation of unaffected partitions
- Faster and scalable distributed processing

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Ans:

```
from pyspark.sql.functions import col

result_df = df.filter(

    (col("status") == "Completed") &

    (col("sale_amount") > 1000)

)

result_df.show()
```

Explanation

- `col("status") == "Completed"` → selects only completed orders
- `col("amount") > 1000` → filters high-value orders
- `&` → combines both conditions (AND logic in Spark)

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Ans:

Predicate Pushdown is an optimization technique used in Parquet (columnar file format) where filter conditions are applied at the data source level during file reading, instead of after loading the entire dataset into memory.

Parquet stores metadata for each row group, such as minimum and maximum values of columns. When a query with a filter condition (for example, $\text{age} > 30$) is executed, Spark uses this metadata to identify and skip irrelevant row groups that do not satisfy the condition.

As a result, only the required portions of the data are read from disk and loaded into memory.

Impact on memory usage:

- Reduces the amount of data loaded into memory
- Avoids reading unnecessary row groups
- Decreases disk I/O operations
- Improves overall query performance and efficiency

Thus, Predicate Pushdown ensures efficient data processing by minimizing memory consumption and reading only relevant data from Parquet files.

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

Ans:

```
from pyspark.sql.functions import col
```

```
df = df.withColumnn(
```

```
    "final_price",
```

```
    col("price") * 1.18
```

```
)
```

```
df.show(5)
```

Output:

user_id	transaction_date	region	product_category	total_sale	city	age	subscription	price	store_id	email	username	status	raw_timestamp	final_price
1001	01-01-2025	West	Electronics	100	Delhi	18.0	Premium	NULL	S101	NULL	NULL	NULL	01-01-2025 10:00	NULL
1002	02-01-2025	West	Furniture	105	Delhi	19.0	Basic	105	S101	user1002@gmail.com	user1002	Completed	02-01-2025 10:01	123.89999999999999
1003	03-01-2025	West	Clothing	110	Delhi	20.0	Premium	110	S101	user1003@gmail.com	user1003	Completed	03-01-2025 10:02	129.79999999999998
1004	04-01-2025	West	Grocery	115	Delhi	21.0	Basic	115	S101	user1004@gmail.com	user1004	Completed	04-01-2025 10:03	135.7
1005	05-01-2025	West	Electronics	120	Delhi	22.0	Premium	120	S101	user1005@gmail.com	user1005	Completed	05-01-2025 10:04	141.6

only showing top 5 rows

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Ans:

Feature	Transformations	Actions
Definition	Operations that create a new DataFrame/RDD from an existing one	Operations that trigger execution of the Spark program
Execution	Lazy (not executed immediately)	Eager (executed immediately)
Purpose	Used to define processing logic	Used to produce final result or output
Return Type	New DataFrame or RDD	Result value or output (not a DataFrame pipeline)
DAG Impact	Adds steps to DAG	Triggers DAG execution

Examples of Transformations

1. filter()

```
df.filter(col("age") > 30)
```

2. withColumn()

```
df.withColumn("new_col", col("price") * 2)
```

Examples of Actions

1. show()

```
df.show()
```

2. count()

```
df.count()
```

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Ans:

```
from pyspark.sql.functions import col
# Load Parquet file
df = spark.read.parquet("path/to/input")
# Filter out rows where user_id is null
clean_df = df.filter(col("user_id").isNotNull())
# Save as CSV
clean_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Explanation:

- `spark.read.parquet()` → reads Parquet file
- `filter(col("user_id").isNotNull())` → removes null `user_id` rows
- `.write.csv()` → saves output in CSV format
- `mode("overwrite")` → replaces existing output folder
- `header=True` → includes column names in CSV

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Ans:

Feature	Client Mode	Cluster Mode
Driver Location	Runs on the machine where the application is submitted (client machine)	Runs inside the cluster (on a worker node)
Execution	Driver communicates with cluster from outside	Driver runs inside cluster
Network Dependency	Requires continuous connection between client and cluster	No dependency on client after submission

Failure Impact	If client disconnects, application may fail	Application continues even if client disconnects
Use Case	Development, debugging, interactive jobs	Production, large-scale jobs
Resource Usage	Uses client machine resources for driver	Uses cluster resources for driver

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Ans:

```
from pyspark.sql.functions import col
```

```
result_df = df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
)
```

```
result_df.show()
```

Explanation:

- `col("region") == "North"` → selects rows from North region
- `col("priority") == "High"` → selects high priority rows
- `|` → OR condition in Spark
- `filter()` → applies condition to DataFrame

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Ans:

Using `.show(5)` is safer because it only displays a **small sample of rows (first 5)** without loading the entire dataset into the driver memory.

In contrast, `.collect()` retrieves **all rows from the distributed cluster to the driver node**, which can cause:

- OutOfMemory error (driver crash)
- Very high network overhead
- Extremely slow performance on large datasets
- Potential system instability for multi-terabyte data

Key Difference

Method	Behavior	Risk
<code>.show(5)</code>	Displays only first 5 rows	Safe
<code>.collect()</code>	Loads entire dataset into driver	Risk of memory crash